

2026 云斗学院软件能力认证第一轮 (YDSP - Junior) 入门级 C++ 语言试题

考生注意事项:

- 试题纸一共 16 页, 满分 100 分。作答后请记录答案, 并按要求提交至对应比赛处。
- 考试过程中不得使用任何电子设备或查阅任何书籍资料。

1 选择题 (每题 2 分, 共 30 分)

1.1 第 1 题

根据 NOI 系列比赛的规则, CSP-J 第二轮考场上, 不可以 ▲。

- A. 进行 Deep sleep
- B. 打开 GUIDE
- C. 自行重启电脑
- D. 食用豆包

1.2 第 2 题

C++ 表达式 `bool(4)+bool(-1)+int(-3.7)` 的值是 ▲。

- A. -4
- B. -3
- C. -2
- D. -1

1.3 第 3 题

现定义数组 `int a[10]`, 想要得到 `a[7]` 的指针, 下列写法正确的是 ▲。

- A. `&(a+7)`
- B. `*a+7`
- C. `a+28`
- D. `a+7`

1.4 第 4 题

令 T 为一棵有根树, 结点 8 的祖先有 1, 2, 3, 5, 9, 结点 5 的祖先有 2, 3, 9。

那么在结点 1, 9, 5, 2 中, 深度最大的结点是 ▲。

- A. 1
- B. 8
- C. 5
- D. 9

1.5 第 5 题

小明有一堆卡片，每张卡片写有 $1 \sim 9$ 。一天上午，他找了一些人，每个人恰好分到一张卡片，并告诉他们，卡片上写着几，就在下午几点把卡片还回来（放到卡片堆的最上面），形成一叠卡片。这样，通过所有人合作，就能给所有卡片排序。该方法最接近于 ▲。

- A. 计数排序
- B. 冒泡排序
- C. 选择排序
- D. 插入排序

1.6 第 6 题

现有一个单向链表，有 n 个结点。链表内有两个结点 p, q ，它们的距离是一个未知数 d （和 n 不同阶）。现在想要判断，**假如**一个指针从链表头开始遍历，会先遍历到 p 还是 q ，能做到的最好复杂度为 ▲。

- A. $O(1)$
- B. $O(d)$
- C. $O(n)$
- D. $O(dn)$

1.7 第 7 题

计算 $2026_8 + 2026_{16} - 10\ 0000\ 0000\ 0000_2$ 的结果为 ▲。

- A. 2026
- B. 1092
- C. 1084
- D. 前三个选项都不对

1.8 第 8 题

考虑如下程序片段：

```
int f(int n,int &m){
    if(n!=0)
        m+=f(n-1,m);
    return n+m;
}

int main(){
    int x=0;
    cout<<f(4,x)<<endl;
}
```

该程序片段的输出为 ▲。

- A. 10
- B. 15
- C. 0

D. 前三个选项都不对

1.9 第 9 题

一棵二叉树的前序遍历为 EBCDAFGH，中序遍历为 CBDFGAEH，则其后序遍历为 ▲。

A. HGFADCBE

B. CFGADBHE

C. HGFADCBE

D. 前三个选项都不对

1.10 第 10 题

字符串 yundou 有 ▲ 个没有重复字母的、长度为 3 的子序列（下标可以不连续）。

A. 15

B. 19

C. 119

D. 前三个选项都不对

1.11 第 11 题

现有一道区间动态规划题，转移方程为

$$f(l, r) = \begin{cases} \max\{f(l, r-1), f(l+1, r)\} + w_{l,r} & 1 \leq l < r \leq n \\ w_{l,r} & 1 \leq l = r \leq n \end{cases}$$

其中 n 为输入的正整数， w 为输入的二维数组，保证 $3 \leq n \leq 500$ ， $1 \leq w_{l,r} \leq 10^6$ 。下列说法正确的是 ▲。

A. 用程序求解动态规划时，无论采用何种写法， $f(2, 8)$ 总是先于 $f(5, 9)$ 被计算出。

B. $f(1, n)$ 可能超过 32 位有符号整数可表示的范围。

C. 求解 $f(1, n)$ 的时间复杂度为 $O(n^2)$ 。

D. 如果把 $w_{1,3}$ 增加 9.9×10^5 （仍然满足 $w_{l,r} \leq 10^6$ ），那么 $f(1, n)$ 至少增加 1。

1.12 第 12 题

在如下代码框中每行选取一条语句，从上到下构成一个程序片段，其输出最大值为 ▲。

```
int x=5;      int x=7;
x+=6;        x*=2;
x-=5;        x-=3;
x*=3;        x/=0.2;
x/=2;        x/=3;
cout<<x;
```

A. 9

B. 27

C. 28

D. 前三个选项都不对

1.13 第 13 题

yummy 正在做一道图论题，需要存储一张 n 个结点 m 条边的有向图。为此，他把结点依次编号为 $1 \sim n$ ，并使用 `vector<int> g[100001]` 来存图。具体地，`g[u]` 存储了 u 所有出边的终点。下列说法正确的是 ▲ 。

- A. `g[u].size()` 记录了结点 u 的度数。
- B. 要判断是否存在一条边 $x \rightarrow y$ ，只需判断 `g[x][y]` 是否存在。
- C. 要在图上加入一条边 $x \rightarrow y$ ，可以写成 `g[x].push_back(y);`。
- D. 如果 $n = 10^5$ ， $m = 5 \times 10^5$ ，那么会发生数组或 `vector` 溢出。

1.14 第 14 题

正整数 m 满足：对于所有整数 $2 \leq x \leq 100$ ，如果 $\gcd(x, m) = 1$ ，那么 x 是素数。那么， m 可以是 ▲ 。

- A. 480
- B. 420
- C. 2026
- D. 前三个选项都不对

1.15 第 15 题

考虑正十边形的十个顶点。任意选择三个顶点可以构成 ▲ 种三角形（全等的三角形看作同一种）。

- A. 8
- B. 12
- C. 6
- D. 120

2 阅读程序（无特殊说明时判断 1.5 分，选择 3 分，3 题共 40 分）

2.1 第 1 题（12 分）

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 unsigned int trans(unsigned int x) {
5     unsigned int p = 1, ans = 0;
6     while (x > 0) {
7         if (x & 1)
8             ans += p;
9         p *= 3;
10        x >>= 1;
11    }
12    return ans;
```

```
13 }
14
15 int main() {
16     int n;
17     cin >> n;
18     while (n--) {
19         unsigned int x;
20         cin >> x;
21         cout << trans(x) << " ";
22     }
23     cout << "\n";
24     return 0;
25 }
```

输入数据满足： $1 \leq n \leq 10$, $0 \leq x \leq 1023$ 。

2.1.1 判断题

1. 对任意合法的 x , $\text{trans}(x)$ 的三进制表示与 x 的二进制表示具有相同的数字序列。
2. 对任意满足 $0 \leq x < 1023$ 的整数 x , 均有 $\text{trans}(x + 1) > \text{trans}(x)$ 。
3. 若 x 的二进制表示中恰有 t 个数位为 1, 则 $\text{trans}(x) = \frac{3^t - 1}{2}$ 。
4. 将第 9 行的 `p *= 3;` 改为 `p <<= 1;`, 对任意合法输入, 程序输出的数列与输入的 x 数列相同。

2.1.2 选择题

5. 输入为 `5\n0 1 2 5 10` 时, 程序输出为 ▲。
A. 0 1 2 10 30
B. 0 1 3 10 30
C. 0 1 3 12 30
D. 0 1 3 10 27
6. 在给定输入范围内, $\text{trans}(x)$ 的最大可能值为 ▲。
A. 19682
B. 29524
C. 59048
D. 59049

2.2 第 2 题 (13 分)

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int N = 200010;
5
6 int n, k;
7
8 struct segment {
9     int l, r;
10 } a[N];
11
12 bool cmp(segment x, segment y) {
13     if (x.r != y.r)
14         return x.r < y.r;
15     return x.l > y.l;
16 }
17
18 bool check(int d) {
19     long long last = -2000000000LL;
20     int cnt = 0;
21     for (int i = 1; i <= n; i++) {
22         if (a[i].l >= last + d) {
23             last = a[i].r;
24             cnt++;
25         }
26     }
27     return cnt >= k;
28 }
29
30 int main() {
31     cin >> n >> k;
32     int mn = 1000000000, mx = 0;
33
34     for (int i = 1; i <= n; i++) {
35         cin >> a[i].l >> a[i].r;
36         mn = min(mn, a[i].l);
37         mx = max(mx, a[i].r);
38     }
39
40     sort(a + 1, a + n + 1, cmp);
41
```

```
42     int L = 1, R = mx - mn, ans = -1;
43     while (L <= R) {
44         int mid = (L + R) / 2;
45         if (check(mid)) {
46             ans = mid;
47             L = mid + 1;
48         } else {
49             R = mid - 1;
50         }
51     }
52
53     cout << ans << "\n";
54     return 0;
55 }
```

输入数据满足： $2 \leq k \leq n \leq 2 \times 10^5$ ， $0 \leq l_i \leq r_i \leq 10^9$ 。

2.2.1 判断题

1. 排序完成后，数组中的线段按右端点从小到大排列；右端点相同时，按左端点从大到小排列。
2. 对任意整数 $d \geq 2$ ，若 `check(d)` 的返回值为真，则 `check(d - 1)` 的返回值也一定为真。

2.2.2 选择题

3. 输入为 5 3\n1 3\n4 5\n7 9\n10 11\n12 15 时，程序输出为 ▲。

- A. -1
- B. 2
- C. 3
- D. 4

4. 令 $C = \max r_i - \min l_i$ ，该程序的时间复杂度为 ▲。

- A. $O(n + \log C)$
- B. $O(n \log n + \log C)$
- C. $O(n \log n + n \log C)$
- D. $O(n^2 \log C)$

5. (4 分) 若将第 22 行的判断条件改为 `a[i].l > last + d`，其余代码不变。对第 3 小题给出的输入，程序输出为 ▲。

- A. -1
- B. 1
- C. 2
- D. 3

2.3 第 3 题 (15 分)

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int N = 1005;
5 int val[N], lc[N], rc[N], sz[N], tot;
6 int height, leaves;
7
8 void insert_node(int &u, int x) {
9     if (u == 0) {
10         u = ++tot;
11         val[u] = x;
12         return;
13     }
14     if (x < val[u])
15         insert_node(lc[u], x);
16     else
17         insert_node(rc[u], x);
18 }
19
20 void dfs(int u, int dep) {
21     if (u == 0)
22         return;
23     height = max(height, dep);
24     dfs(lc[u], dep + 1);
25     dfs(rc[u], dep + 1);
26     sz[u] = sz[lc[u]] + sz[rc[u]] + 1;
27     if (lc[u] == 0 && rc[u] == 0)
28         leaves++;
29 }
30
31 int find_node(int u, int x) {
32     while (u != 0) {
33         if (val[u] == x)
34             return u;
35         if (x < val[u])
36             u = lc[u];
37         else
38             u = rc[u];
39     }
40     return 0;
41 }
```



```
42
43 int main() {
44     int n, q, root = 0;
45     cin >> n >> q;
46     for (int i = 1; i <= n; i++) {
47         int x;
48         cin >> x;
49         insert_node(root, x);
50     }
51
52     dfs(root, 1);
53
54     while (q--) {
55         int x;
56         cin >> x;
57         int u = find_node(root, x);
58         cout << (u == 0 ? 0 : sz[u]) << " ";
59     }
60     cout << "\n";
61     cout << height << " " << leaves << "\n";
62     return 0;
63 }
```

输入数据满足： $1 \leq n \leq 1000$, $1 \leq q \leq 1000$, 所有输入整数均在 `int` 范围内。

2.3.1 判断题

1. 若插入的 n 个数互不相同且严格递增, 则程序第二行输出为 $n-1$ 。
2. 交换 `dfs` 函数中两次递归调用的先后顺序, 程序最终输出可能发生改变。

2.3.2 选择题

3. (2分) 输入为 $7\ 3\ 5\ 3\ 8\ 2\ 4\ 7\ 9\ 3\ 8\ 6$ 时, 程序输出为 ▲。
A. $3\ 3\ 0$ 与 $3\ 4$
B. $3\ 3\ 0$ 与 $4\ 3$
C. $2\ 2\ 0$ 与 $3\ 4$
D. $3\ 3\ 1$ 与 $3\ 4$
4. 输入为 $6\ 2\ 4\ 2\ 6\ 2\ 3\ 5\ 2\ 4$ 时, 程序输出为 ▲。
A. $2\ 6$ 与 $4\ 2$
B. $3\ 6$ 与 $4\ 2$
C. $3\ 5$ 与 $3\ 3$

D. 3 6 与 3 2

5. 假设输入序列中整数 x 至少出现一次。程序执行完建树操作后，令

```
int u = find_node(root, x);
```

则关于结点 u ，下列说法中一定正确的是 ▲ 。

- A. u 一定是所有值为 x 的结点中最后插入的结点
- B. u 的左子树中不存在值为 x 的结点
- C. u 的右子树中一定存在值为 x 的结点
- D. u 一定没有左儿子

6. (4 分) 若希望在 `dfs` 函数执行过程中，将二叉搜索树中所有结点的 `val` 按非递减顺序输出，则应在下列哪个位置加入语句 `cout << val[u] << " "`；。相关代码修改如下：

```
void dfs(int u, int dep) {  
    if (u == 0)  
        return;  
    height = max(height, dep);  
  
    // Position A  
    dfs(lc[u], dep + 1);  
  
    // Position B  
    dfs(rc[u], dep + 1);  
  
    // Position C  
    sz[u] = sz[lc[u]] + sz[rc[u]] + 1;  
  
    // Position D  
    if (lc[u] == 0 && rc[u] == 0)  
        leaves++;  
}
```

应加入的位置是 ▲ 。

- A. Position A
- B. Position B
- C. Position C
- D. Position D

3 完善程序（共两道题 30 分）

3.1 数轴上的饼干（15 分）

数轴上有 n 块饼干，第 i 块饼干所在位置的坐标为 a_i ，所有 a_i 两两不同。

小 C 初始位于坐标 0。只要还有饼干没有被取走，他就重复下面的操作：

- 在所有尚未取走的饼干中，选择与当前位置距离最近的一块；
- 如果有两块饼干与当前位置的距离相同，则选择**坐标较小**的那一块；
- 移动到该饼干所在位置并将其取走。

请计算小 C 取走全部 n 块饼干时移动的总距离。

输入满足 $1 \leq n \leq 10^5$ ， $-10^9 \leq a_i \leq 10^9$ 。试补全程序。

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7
8     vector<long long> a(n);
9     for (int i = 0; i < n; i++)
10         cin >> a[i];
11
12     sort(a.begin(), a.end());
13
14     int r = 0;
15     while (/* Blank 1 */)
16         r++;
17     int l = r - 1;
18
19     long long pos = 0;
20     long long ans = 0;
21
22     while (/* Blank 2 */) {
23         bool goLeft =
24             /* Blank 3 */;
25
26         if (goLeft) {
27             ans += pos - a[l];
28             /* Blank 4 */;
29         } else {
30             ans += a[r] - pos;
```

```
31         /* Blank 5 */;  
32     }  
33 }  
34  
35     cout << ans << "\n";  
36     return 0;  
37 }
```

1. /* Blank 1 */ 应填 ▲。

- A. `r < n && a[r] < 0`
- B. `r < n && a[r] > 0`
- C. `r > 0 && a[r] < 0`
- D. `r <= n && a[r] < 0`

2. /* Blank 2 */ 应填 ▲。

- A. `l >= 0 && r < n`
- B. `l >= 0 || r < n`
- C. `l < 0 || r >= n`
- D. `l >= 0 && r >= n`

3. /* Blank 3 */ 应填 ▲。

- A. `l >= 0 && (r >= n || pos - a[l] <= a[r] - pos)`
- B. `l >= 0 && (r >= n || pos - a[l] < a[r] - pos)`
- C. `r < n && (l < 0 || pos - a[l] <= a[r] - pos)`
- D. `l >= 0 && (r >= n || -a[l] <= a[r])`

4. /* Blank 4 */ 应填 ▲。

- A. `pos = a[--l]`
- B. `pos = a[l--]`
- C. `pos = a[l++]`
- D. `pos = a[r--]`

5. /* Blank 5 */ 应填 ▲。

- A. `pos = a[++r]`
- B. `pos = a[r--]`
- C. `pos = a[r++]`
- D. `pos = a[--r]`

3.2 拆墙迷宫 (15 分)

有一个 $n \times m$ 的迷宫，其中 $.$ 表示可以通过的空地， $\#$ 表示墙， S 表示起点， T 表示终点。每一步可以向上、下、左、右移动到相邻格子。

你至多可以选择一堵墙，在第一次走到这堵墙时将它拆掉，并进入这个格子；拆掉后该格子可正常通过。求从 S 到 T 的最少步数。若无法到达，输出 -1 。

输入满足 $1 \leq n, m \leq 100$ ，且迷宫中恰有一个 S 和一个 T 。试补全程序。

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Node {
5     int x, y, used;
6 };
7
8 int n, m;
9 char g[105][105];
10 int dista[105][105][2];
11 int dx[4] = {-1, 1, 0, 0};
12 int dy[4] = {0, 0, -1, 1};
13
14 int main() {
15     cin >> n >> m;
16
17     int sx, sy, tx, ty;
18     for (int i = 0; i < n; i++) {
19         cin >> g[i];
20         for (int j = 0; j < m; j++) {
21             if (g[i][j] == 'S') sx = i, sy = j;
22             if (g[i][j] == 'T') tx = i, ty = j;
23         }
24     }
25
26     /* Blank 1 */;
27
28     queue<Node> q;
29     dista[sx][sy][0] = 0;
30     q.push({sx, sy, 0});
31
32     while (/* Blank 2 */) {
33         Node u = q.front();
34         q.pop();
35
36         for (int d = 0; d < 4; d++) {
```

```
37         int nx = u.x + dx[d];
38         int ny = u.y + dy[d];
39
40         if (nx < 0 || nx >= n || ny < 0 || ny >= m)
41             continue;
42
43         if (/* Blank 3 */)
44             continue;
45
46         int nused =
47             /* Blank 4 */;
48
49         if (dista[nx][ny][nused] != -1)
50             continue;
51
52         dista[nx][ny][nused] =
53             dista[u.x][u.y][u.used] + 1;
54         q.push({nx, ny, nused});
55     }
56 }
57
58 int ans = dista[tx][ty][0];
59 if (dista[tx][ty][1] != -1)
60     /* Blank 5 */;
61
62 cout << ans << "\n";
63 return 0;
64 }
```

1. /* Blank 1 */ 应填 ▲。

- A. fill(dista[0][0], dista[0][0] + 2, -1)
- B. memset(g, -1, sizeof(g))
- C. memset(dista, 0, sizeof(dista))
- D. memset(dista, -1, sizeof(dista))

2. /* Blank 2 */ 应填 ▲。

- A. !q.empty()
- B. dista[tx][ty][0] == -1
- C. q.empty()
- D. q.size() == 1

3. /* Blank 3 */ 应填 ▲。

- A. `g[nx][ny] == '#' && u.used == 0`
- B. `g[nx][ny] != '#' && u.used == 1`
- C. `g[nx][ny] == '#' && u.used == 1`
- D. `g[nx][ny] == '#'`

4. /* Blank 4 */ 应填 ▲。

- A. `u.used + (g[nx][ny] == '#')`
- B. `u.used`
- C. `u.used - (g[nx][ny] == '#')`
- D. `g[nx][ny] == '#'`

5. /* Blank 5 */ 应填 ▲。

- A. `ans = min(ans, dista[tx][ty][1])`
- B. `ans = (ans == -1 ? dista[tx][ty][1] : min(ans, dista[tx][ty][1]))`
- C. `ans = (ans == -1 ? dista[tx][ty][1] : max(ans, dista[tx][ty][1]))`
- D. `ans = (ans == -1 ? -1 : min(ans, dista[tx][ty][1]))`

--	--	--	--	--	--	--	--

0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1
2	2	2	2	2	2	2	2
3	3	3	3	3	3	3	3
4	4	4	4	4	4	4	4
5	5	5	5	5	5	5	5
6	6	6	6	6	6	6	6
7	7	7	7	7	7	7	7
8	8	8	8	8	8	8	8
9	9	9	9	9	9	9	9

1

A

B

C

D

2

A

B

C

D

3

A

B

C

D

4

A

B

C

D

5

A

B

C

D

6

A

B

C

D

7

A

B

C

D

8

A

B

C

D

9

A

B

C

D

10

A

B

C

D

11

A

B

C

D

12

A

B

C

D

13

A

B

C

D

14

A

B

C

D

15

A

B

C

D

16

T

F

X

X

17

T

F

X

X

18

T

F

X

X

19

T

F

X

X

20

A

B

C

D

21

A

B

C

D

22

T

F

X

X

23

T

F

X

X

24

A

B

C

D

25

A

B

C

D

26

A

B

C

D

27

T

F

X

X

28

T

F

X

X

29

A

B

C

D

30

A

B

C

D

31

A

B

C

D

32

A

B

C

D

33

A

B

C

D

34

A

B

C

D

35

A

B

C

D

36

A

B

C

D

37

A

B

C

D

38

A

B

C

D

39

A

B

C

D

40

A

B

C

D

41

A

B

C

D

42

A

B

C

D